

PROYECTO ACADÉMICO / IONIC + ANGULAR

Gestor de tareas

Paso a paso del desarrollo de una app móvil híbrida para organizar deberes de estudio.

OBJETIVO

Crear una aplicación que permita registrar tareas, asignar prioridad, completarlas y conservar la lista al volver a abrirla.

Qué incluye

- Interfaz móvil hecha con componentes de Ionic.
- Formulario reactivo con validación y selección de prioridad.
- Servicio de Angular para administrar y guardar tareas.
- Actualización inmediata de la lista sin recargar el navegador.

El código fuente, las instrucciones de ejecución y este documento forman parte de la entrega.

1. Preparar el entorno

Ionic utiliza Node.js y npm para crear el proyecto y descargar sus dependencias.

Instalar Node.js y npm

Instala Node.js LTS desde nodejs.org. npm se instala junto con Node.js. Abre una terminal nueva y confirma que ambos comandos respondan:

```
node --version  
npm --version
```

Instalar el CLI de Ionic

```
npm install -g @ionic/cli  
ionic --version
```

Crear el proyecto

Se utilizó la plantilla vacía de Ionic para Angular. Capacitor se integró como base para poder preparar versiones móviles más adelante.

```
ionic start gestor-tareas blank --type=angular --no-interactive  
cd gestor-tareas
```

La plantilla generó la configuración de Angular, el router, estilos globales y estructura inicial de Ionic.

2. Modelar y guardar las tareas

Los datos y las operaciones se concentran en el modelo TypeScript y un servicio de Angular.

Modelo `Tarea`

Cada tarea tiene un identificador, título, descripción, prioridad y estado completado. El tipo `Prioridad` restringe las opciones a Alta, Media y Baja.

```
export type Prioridad = 'Alta' | 'Media' | 'Baja';

export interface Tarea {
  id: string;
  titulo: string;
  descripcion: string;
  prioridad: Prioridad;
  completada: boolean;
}
```

Servicio `TareasService`

El servicio está disponible en toda la aplicación mediante `@Injectable({ providedIn: 'root' })`. Un `BehaviorSubject` conserva el arreglo actual y comunica cada cambio a la pantalla.

- `agregar()` crea una tarea pendiente con un identificador único.
- `alternarCompletada()` cambia su estado.
- `eliminar()` la quita de la lista.
- `guardar()` escribe el nuevo arreglo en el navegador y emite el cambio.

Persistencia local

Al iniciar, el servicio lee la clave `gestor-tareas` de `localStorage`. Esto conserva los datos en el mismo navegador y dispositivo; no sincroniza entre dispositivos.

```
const guardadas = localStorage.getItem('gestor-tareas');
return guardadas ? JSON.parse(guardadas) : [];
```

3. Diseñar la pantalla y el formulario

La pantalla principal está en `src/app/home/` y usa componentes de Ionic adaptados a una vista móvil.

- `ion-header`, `ion-title`, `ion-content` e `ion-footer` organizan la pantalla.
- `ion-list`, `ion-item` e `ion-label` presentan la lista de tareas.
- `ion-badge` identifica la prioridad con color.
- `ion-checkbox` completa una tarea y el título aparece tachado.
- Un botón con papelera y el gesto de deslizar permiten eliminarla.
- `ion-fab` abre el formulario; filtros y resumen muestran el progreso.

Formulario reactivo

`TaskFormComponent` usa `ReactiveFormsModule` y un `FormGroup`. El título es obligatorio y requiere al menos cinco caracteres sin contar espacios externos. La descripción es opcional y la prioridad comienza en Media.

```
titulo: ['', [Validators.required, tituloMinimo]],
descripcion: ['', ],
prioridad: ['Media', Validators.required]
```

Paso de datos entre componentes

Al guardar, el modal devuelve título, descripción y prioridad a `HomePage` con el rol `confirm`. La pantalla principal entrega esos datos al servicio. Si el usuario cierra o cancela el modal, no se agrega ninguna tarea.

Actualización inmediata

La pantalla conecta el estado del servicio con Angular mediante `toSignal`. La respuesta del cierre del modal de Ionic se procesa dentro del contexto de Angular y se actualiza la vista. Así la tarea aparece al guardarla, sin recargar el navegador.

Los colores del formulario se fijaron para mantener el contraste en modo claro u oscuro. El espaciado, las tarjetas y los distintivos hacen más clara la lectura en teléfono.

4. Instalar dependencias y ejecutar

Usa estos comandos desde la carpeta del proyecto después de clonarlo o descargarlo.

Primera ejecución

```
cd gestor-tareas
npm install
ionic serve
```

`npm install` descarga las dependencias indicadas por `package-lock.json`. `ionic serve` inicia el servidor de desarrollo y abre la app en el navegador. El servidor actualiza la vista cuando se guardan cambios en el código. Para detenerlo, presiona `Ctrl+C`.

Compilar la app web

```
ionic build
```

La compilación de producción se ejecutó correctamente. Sus archivos web se generan dentro de `www/`.

Archivos principales

- `src/app/models/tarea.model.ts`: modelo de datos.
- `src/app/services/tareas.service.ts`: operaciones y persistencia.
- `src/app/home/`: pantalla principal, lista y filtros.
- `src/app/task-form/`: formulario reactivo y validaciones.

Contenido de entrega

- Repositorio GitHub con el código fuente y `README.md`.
- Este PDF con los pasos de desarrollo.
- No se incluye `node\_modules`; se crea con `npm install`.

La app usa Ionic y Angular en el navegador, con Capacitor como base para empaquetar una versión nativa cuando se agregue una plataforma móvil.